

Data analysis and visualization

Final Architecture & FinOps Report

Topic : Pakistan Weather Analytics Lakehouse Pipeline

(Lahore, Karachi, Islamabad, Peshawar, Quetta)

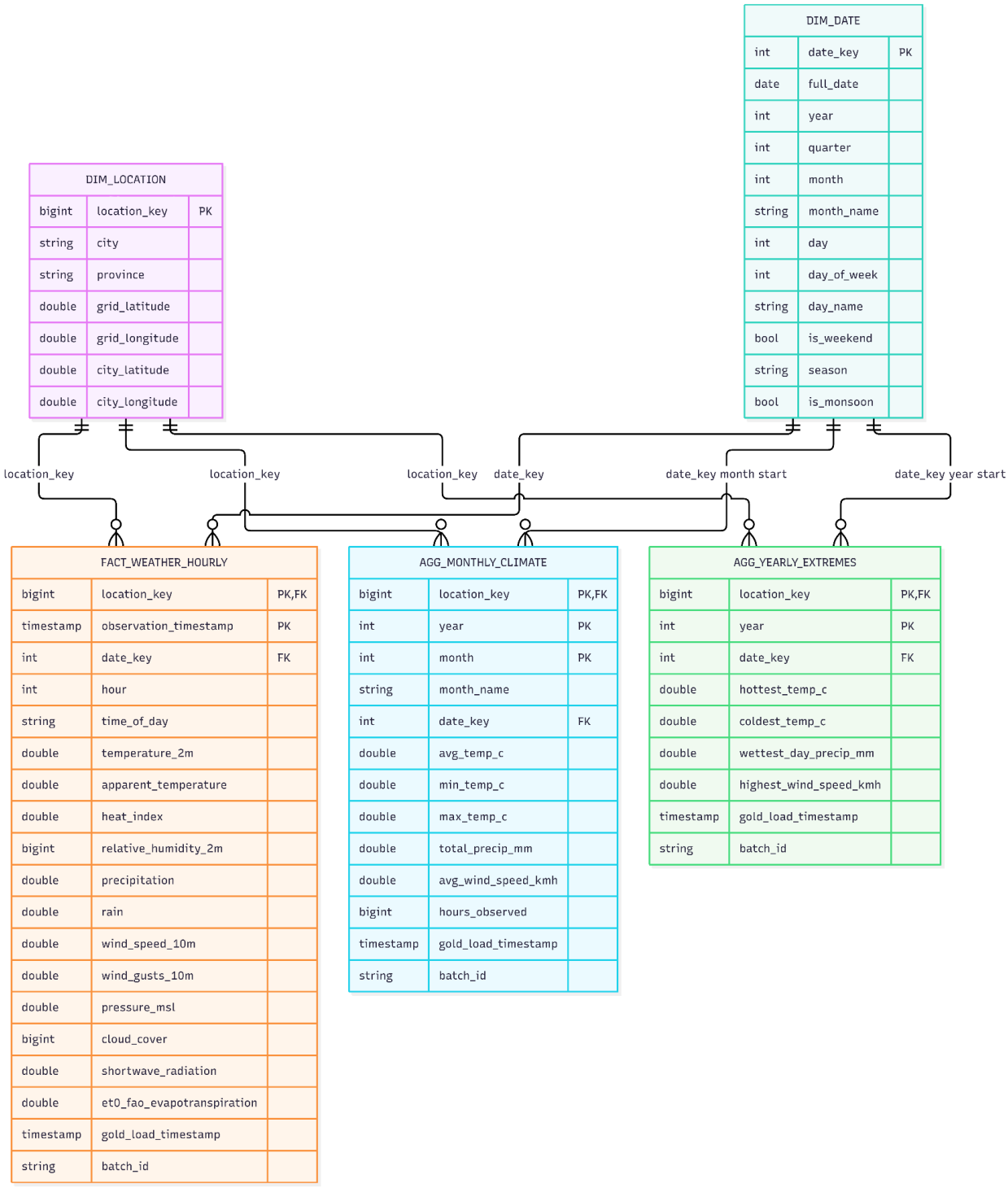
BDS - 5B

Group members

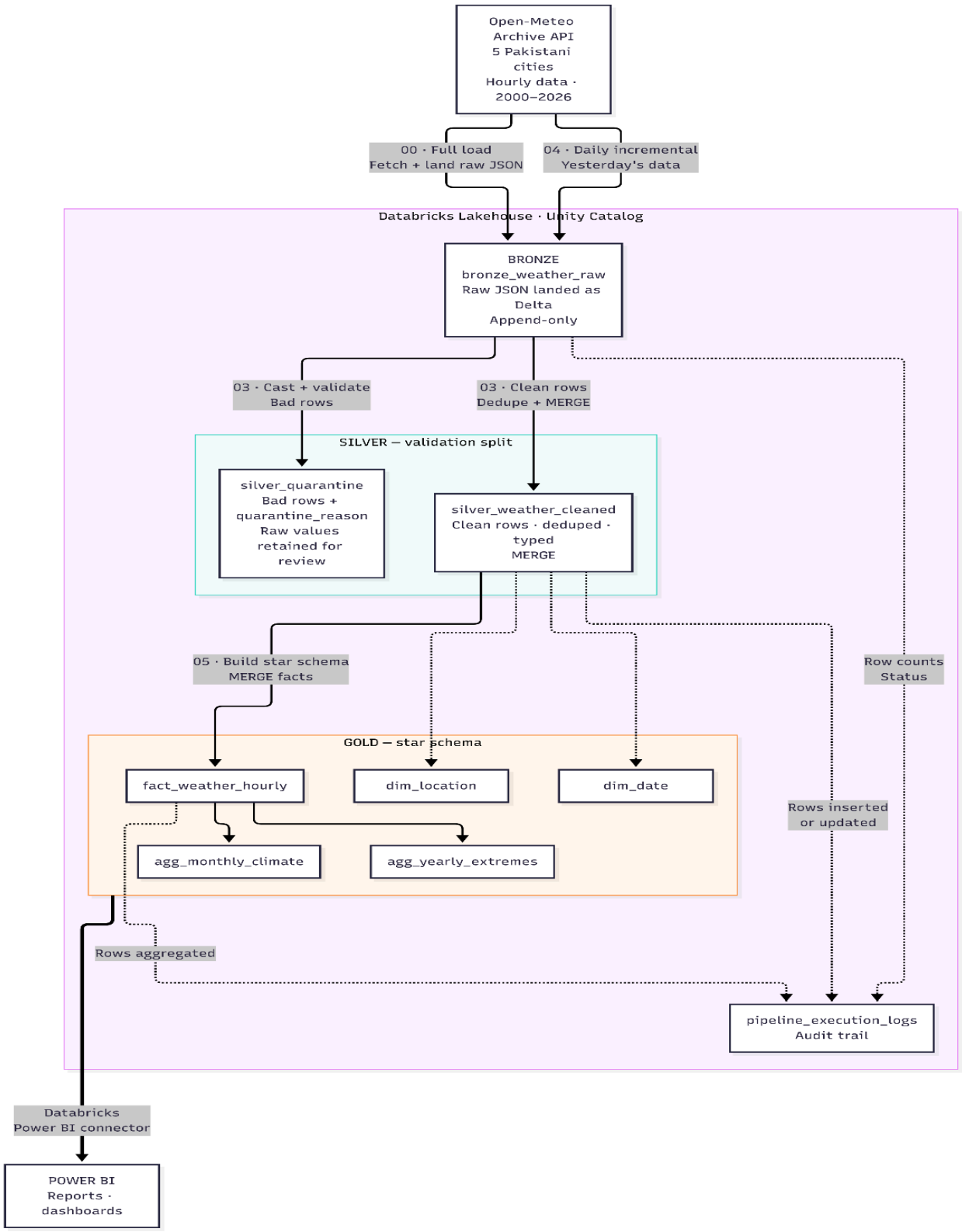
Syeda Faryal Zehra	24L-2556
Ibrahim Qaiser	24L-2513

Pakistan 5 cities Weather Analytics Pipeline

ERD Diagram for Gold Layer



Section A: Architectural Diagram



Section B: Decision Log : Defending Engineering choices

Partitioning / Clustering: We didn't use old-style Hive partitioning. We used Delta liquid clustering on (location_key, date_key) for the fact table, because every query — the monthly and yearly aggregates, validation checks, and Power BI dashboards — always filters by city and date. Liquid clustering targets exactly those columns, adapts automatically as the data grows, and avoids the small-file and over-partitioning problems fixed partitions cause on 24+ years of hourly data.

Schema Drift: The API sometimes added or changed fields. We handled it in stages: Bronze ingests with schema evolution enabled and a rescued_data column, so unknown new fields are rescued instead of breaking the load. Silver validates and casts every column — rows that fail go to a separate silver_quarantine table with a quarantine_reason, so bad data never silently reaches Gold. Gold uses MERGE (idempotent — re-runs never duplicate) and rebuilds dimensions with overwriteSchema when the calendar or cities change.

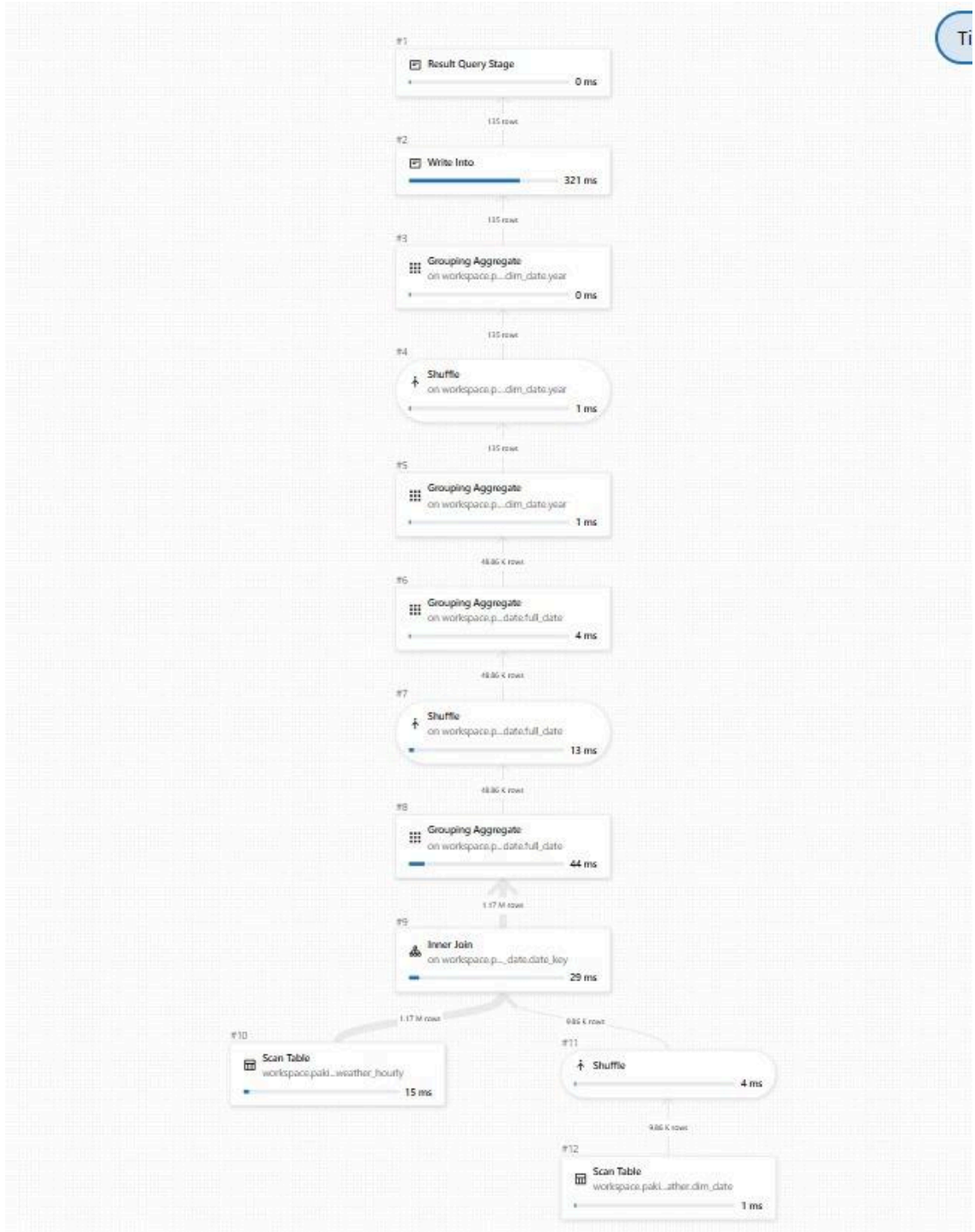
PII: Our dataset has no personal data — it's public weather readings for 5 city grid points, so there is nothing to mask. The design still protects data access through Unity Catalog permissions, and if PII ever entered the pipeline, we would apply UC column masks and row filters before Silver.

Section C: Spark UI & Query Plan DAG Analysis

Selected Execution Cell: Cell 7 (agg_yearly_extremes) — Selected as it represents the pipeline's most complex multi-stage aggregation over the primary fact table.

Execution Breakdown:

- Table Scans & File Pruning (Nodes #10 & #12): Spark scans the base Delta storage layer, reading 1.17M rows from fact_weather_hourly and 9.86K rows from dim_date while using Delta metadata for file pruning.
- Dimension Join (Node #9 — Inner Join): Executes an inner join between the hourly fact table and dim_date on date_key to append temporal calendar features.
- Stage 1 Daily Aggregation (Nodes #8, #7, #6): Executes a Partial Grouping Aggregate locally on worker nodes before triggering a Shuffle Exchange across full_date, reducing 1.17M hourly rows down to 48.86K daily summary records.
- Stage 2 Yearly Aggregation (Nodes #5, #4, #3): Performs a second Shuffle Exchange on dim_date.year and applies a final Hash Aggregate to distill the 48.86K daily records into 135 final yearly climate extreme metrics.
- Delta Target Write (Node #2): Idempotently writes the final 135 rows into the agg_yearly_extremes


#1
Result Query Stage
0 ms
135 rows
#2
Write Into
321 ms
135 rows
#3
Grouping Aggregate
on workspace.p...dim_date.year
0 ms
135 rows
#4
Shuffle
on workspace.p...dim_date.year
1 ms
135 rows
#5
Grouping Aggregate
on workspace.p...dim_date.year
1 ms
48.86 K rows
#6
Grouping Aggregate
on workspace.p...date.full_date
4 ms
48.86 K rows
#7
Shuffle
on workspace.p...date.full_date
13 ms
48.86 K rows
#8
Grouping Aggregate
on workspace.p...date.full_date
44 ms
1.17 M rows
#9
Inner Join
on workspace.p..._date.date_key
29 ms
1.17 M rows
9.86 K rows
#10
Scan Table
workspace.paki...weather_hourly
15 ms
#11
Shuffle
4 ms
9.86 K rows
#12
Scan Table
workspace.paki...ather.dim_date
1 ms

Section D: Cost & FinOps Awareness Evaluation

1. Baseline Environment & Execution Metrics

- **Execution Environment:** Databricks Serverless / Free Tier Compute (Azure for Students / Databricks Community Edition).
- **Incremental Daily Run Duration: ~30 to 45 seconds** ($0.0125 hours$) total execution across Bronze, Silver, and Gold notebooks.
- **Full Pipeline Reload Duration: ~400 to 550 seconds** ($0.125 hours$) when executing full historical backfills and Delta table compaction.

2. Enterprise Cost Model & Projections

Assuming an enterprise production deployment running on a standard 3-node cluster or Serverless SQL/Jobs compute costing **~$1.20 per compute-hour** (VM + Databricks DBUs):

- **Cost Per Incremental Run:**

  Cost = 0.0125 x $1.20/hour= $0.015 (per run)

- **Annualized Hourly Execution (8,760 runs/year):**
  $RunsperYear = 24 runs/day \times 365 days = 8,760 runs$

  Annual Compute Cost = 8,760 X $0.015 =$131.40 / year

3. Architectural Cost Optimization & FinOps Controls

- **Selective OPTIMIZE Execution Control**
  - Delta file compaction via OPTIMIZE and Liquid Clustering is conditionally gated strictly to full reloads (if LOAD_TYPE == 'full':). Running file compaction during light hourly or daily incremental loads would inflate runtimes by **~10×**, escalating annual compute spend from **~$131.40/year** to over **~$1,300/year** without meaningful performance gains.
- **Memory-Optimized Broadcast Hash Joins**
  - Dimension tables (dim_location and dim_date) are explicitly broadcasted (F.broadcast()) into executor node memory during joins. This eliminates costly network shuffles of the multi-million row fact table (fact_weather_hourly), drastically reducing CPU and memory overhead.
- **Delta Lake Metadata Pruning & Clustering**
  - Structuring fact_weather_hourly with liquid clustering/partitioning on (location_key, date_key) allows Spark and Power BI engine queries to skip unneeded Parquet files via file pruning, preventing expensive full table scans.